FACIAL EMOTION DETECTION SYSTEM

DEVELOPMENT PHASE / REFLECTION PHASE

```
====================================
     EMOTION DETECTION
====================================
1. Detect Emotion (Single Image)
2. Detect Emotions (Batch Folder)
3. View Past Results
4. Exit
Select option:
```

PROJECT OVERVIEW

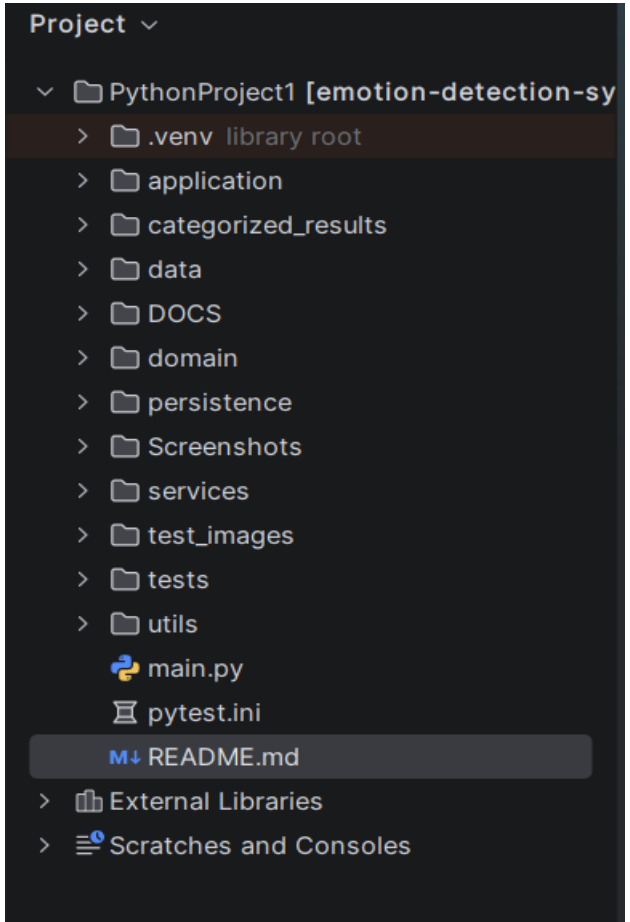
Python-based emotion detection system that analyzes facial expressions from images.

Core Capabilities

- Detects emotions from single images
- Batch processing for entire folders
- Displays confidence scores
- Categorizes images into emotion-based folders
- Logs results into CSV for analysis

Goal

- Automate emotion recognition
- Provide structured output
- Support testing, evaluation, and analytics



FRAMEWORKS USED

Deep Learning and Emotion Recognition

- **TensorFlow / Keras** — backend for emotion model inference https://www.tensorflow.org/
- **DeepFace** — high-level facial emotion recognition wrapper https://github.com/serengil/deepface

Image Processing and Visualization

- **OpenCV** — image loading, preprocessing, color conversion https://opencv.org/
- **Matplotlib** — displays detected emotion + confidence https://matplotlib.org/

Data Storage and File Management

- **CSV module** — persistent logging of results https://docs.python.org/3/library/csv.html (docs.python.org in Bing)
- **OS module** — folder creation, path handling https://docs.python.org/3/library/os.html (docs.python.org in Bing)
- **Shutil module** — copying images into categorized folders https://docs.python.org/3/library/shutil.html (docs.python.org in Bing)

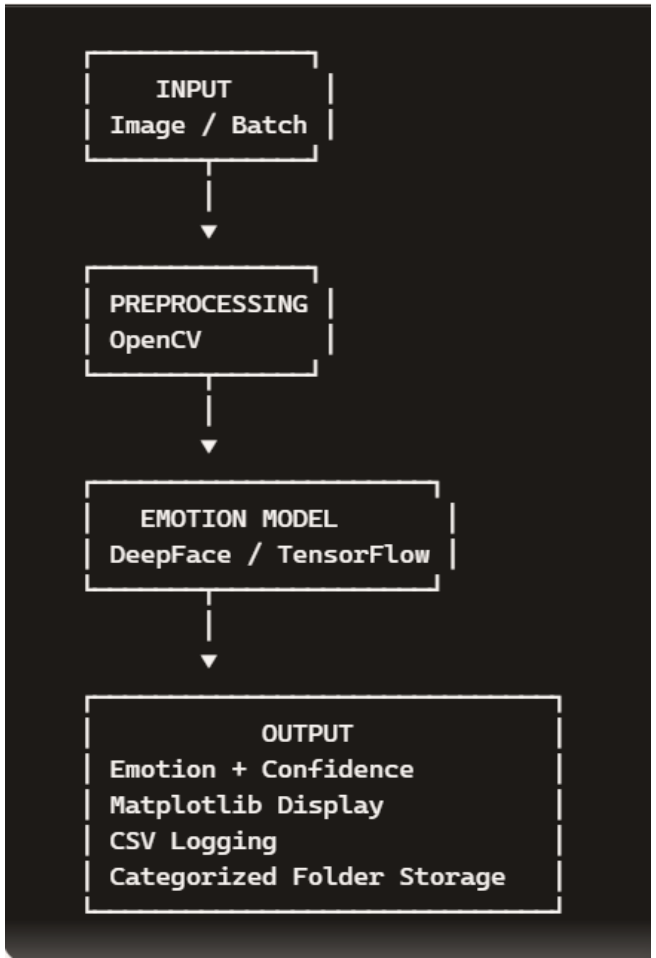
HOW THE FRAMEWORKS WORK TOGETHER

Structured Workflow

- **OpenCV** loads the image and converts BGR → RGB.
- **DeepFace/TensorFlow** analyze the face and return emotion + confidence.
- **Matplotlib** displays the image with prediction overlay.
- **CSV module** logs the result (image path, emotion, confidence).
- **OS + Shutil** create emotion folders and copy processed images into them.

Why this structure works

- Clear separation of responsibilities
- Easy to maintain and extend
- Supports both single and batch processing
- Lightweight and efficient



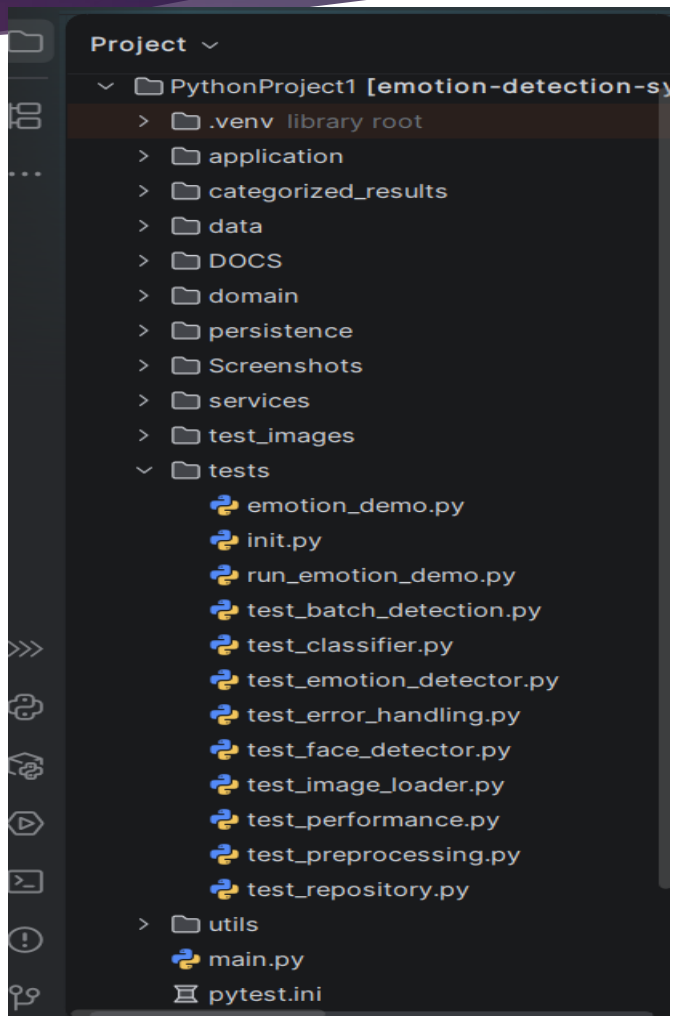
CURRENT ARCHITECTURE

Design Features

- Modular structure
- Clear separation of concerns
- Easy to maintain and extend

Modules

- main.py — CLI workflow
- services/emotion_detector.py — model inference
- persistence — CSV logging
- categorized_results — output storage
- test_images — input dataset



EMOTION DETECTION LOGIC

Detection Output

Each detection returns:

- Emotion label (angry, happy, sad, neutral, surprise, fear, disgust)
- Confidence score
- Original image path

CSV Log Structure

- image_path
- emotion
- confidence

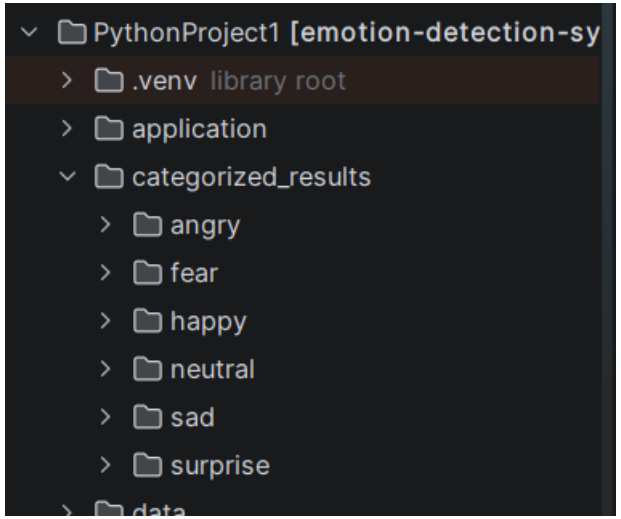
STORAGE APPROACH AND DATA FLOW

Storage Approach

- CSV file for persistent logging
- Folder-based categorization for processed images

Why this approach?

- Lightweight
- Easy to inspect
- No database overhead
- Ideal for batch processing
- **Data Flow** Image → Preprocessing → Model → Result → CSV + Categorized Folder



CLI WORKFLOW

User Interaction Flow

- Select menu option
- Provide image or folder path
- System detects emotion
- Image displayed with Matplotlib
- Result saved to CSV
- Image copied to categorized folder

Menu Options

- Detect single image
- Detect batch folder
- View past results
- Exit

```
-----------------------------------
1. Detect Emotion (Single Image)
2. Detect Emotions (Batch Folder)
3. View Past Results
4. Exit
Select option: 1
Enter image path: test_images/image1.jpg
2026-03-20 09:57:42.331859: I tensorflow/core/platform/cpu_featu
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX

Detected Emotion: angry
Confidence: 80.94%
Copied image to: categorized_results\angry\angry_80_image1.jpg
```

640x480 PNG (32-bit color) 196.54 kB

MODEL AND DETECTION PIPELINE

Pipeline Steps

- Load image with OpenCV
- Convert BGR → RGB
- Pass through emotion model
- Receive emotion + confidence
- Display with Matplotlib

Model Source

- DeepFace backend (or custom CNN)
- Pretrained emotion recognition model

```
1. Detect Emotion (Single Image)
2. Detect Emotions (Batch Folder)
3. View Past Results
4. Exit
Select option: 1
Enter image path: test_images/image2.jpg

Detected Emotion: angry
Confidence: 97.62%
Copied image to: categorized_results\angry\angry_97_image2.jpg
```

Plots

640x480 PNG (32-bit color) 235.38 kB

BATCH PROCESSING AND CATEGORIZATION

Batch Mode Features

- Processes all images in a folder
- Prints formatted table of results
- Copies each image into correct emotion folder
- Adds confidence to filename to avoid duplicates

```
Select option: 2
Enter folder path: test_images

Processing 15 image(s)...

File                          Emotion        Confidence
--------------------------------------------------------
image1.jpg                    angry          80.94%
Copied to: categorized_results\angry\angry_80_image1.jpg
image10.jpg                   surprise       59.32%
Copied to: categorized_results\surprise\surprise_59_image10.jpg
image11.jpg                   surprise       70.27%
Copied to: categorized_results\surprise\surprise_70_image11.jpg
image12.jpg                   surprise       99.48%
Copied to: categorized_results\surprise\surprise_99_image12.jpg
image13.jpg                   happy          96.79%
Copied to: categorized_results\happy\happy_96_image13.jpg
image14.jpg                   happy          98.49%
Copied to: categorized_results\happy\happy_98_image14.jpg
image15.jpg                   happy          100.00%
Copied to: categorized_results\happy\happy_100_image15.jpg
image2.jpg                    angry          97.62%
Copied to: categorized_results\angry\angry_97_image2.jpg
```

TESTING STRATEGY AND RESULTS

Testing Approach

- Manual testing via CLI
- Automated testing using pytest
- Performance tests
- Validation using multiple test images

Test Cases

- Single image detection
- Batch folder detection
- CSV logging
- Categorization accuracy

Results

- Predictions stable
- Categorization correct
- Performance: ~0.23 sec per image
- CSV logging reliable

```
✓ 1 test passed  1 test total, 3 sec 526 ms

C:\Users\mista\PycharmProjects\PythonProject1\.venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2025.3.3/plugins/py
Testing started at 01:20 ...
Launching pytest with arguments C:\Users\mista\PycharmProjects\PythonProject1\tests\test_performance.py --no-header --no-summa

============================= test session starts ==============================
collecting ... collected 1 item

tests/test_performance.py::test_performance

============================== 1 passed in 15.61s ==============================
PASSED                                                                   [100%]
Images processed: 15
Total time: 3.5220630168914795
Average time per image: 0.23480420112609862
2026-03-15 01:20:31.812774: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use av
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with t

Process finished with exit code 0
```

PROGRESS VS ORIGINAL PLAN

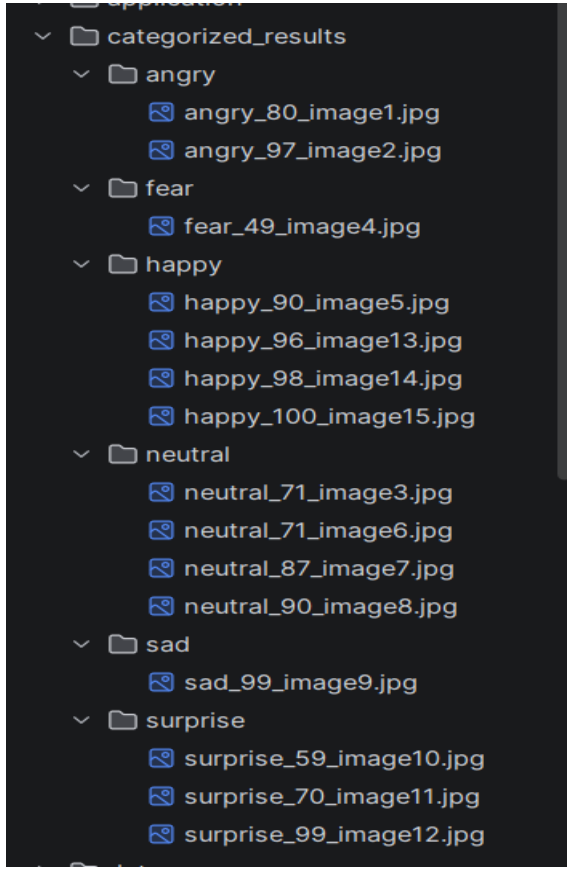
Planned (Phase 1)

- Basic emotion detection
- Single image input
- Simple console output

Achieved (Phase 2)

- ✓ Full CLI menu
- ✓ Single + batch processing
- ✓ Categorized output folders
- ✓ CSV logging
- ✓ Image visualization
- ✓ Performance testing
- **Improvement Summary:**

The project evolved from a simple detector → a structured, multi-feature emotion analysis system.



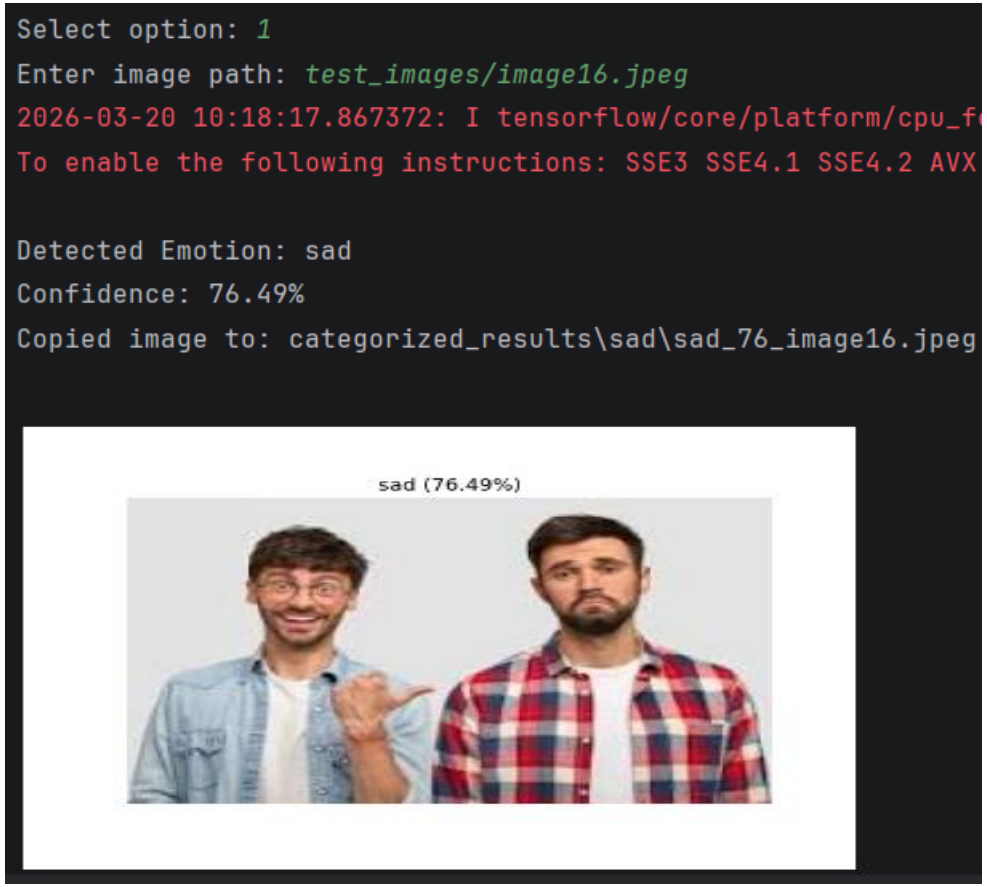
REFLECTION AND FUTURE IMPROVEMENTS

Key Learnings

- Modular Python design
- Image preprocessing
- Deep learning model integration
- CLI design
- Performance optimization

Future Improvements

- One face per image
- GUI (Tkinter or PyQt)
- Web interface (Flask/FastAPI)
- Database storage
- Real-time webcam detection
- More advanced models (Vision Transformers)



Conclusion: The Emotion Detection System demonstrates how Python and machine learning can be combined to create a functional and